UNIC

SISTEMAS DE COMPUTAÇÃO E DE INFORMAÇÃO

DAVID GABRIEL DA SILVA KUHN

LUCAS COLOGNESE MATOS

GUILHERME HENRIQUE GELLI SOARES

ARQUITETURA, DESENVOLVIMENTO DE SOFTWARE, LINGUAGENS, SGBDs E FERRAMENTAS CASE

Resolução de 50 exercícios

Professor: Felipe

Cuiabá

2026

SUMÁRIO

1 INTRODUÇÃO 2

2 ARQUITETURA DE SISTEMAS 3

3 PROCESSOS DE DESENVOLVIMENTO 9

4 LINGUAGENS DE PROGRAMAÇÃO 14

5 SISTEMAS DE GERENCIAMENTO DE BANCOS DE DADOS 18

6 FERRAMENTAS CASE E PROJETO INTEGRADOR 23

7 CONSIDERAÇÕES FINAIS 35

REFERÊNCIAS 36

1 INTRODUÇÃO

Este trabalho apresenta a resolução de 50 exercícios da disciplina Sistemas de Computação e de Informação, abordando arquitetura de software, processos de desenvolvimento, linguagens de programação, sistemas de gerenciamento de bancos de dados (SGBDs) e ferramentas de engenharia de software auxiliada por computador (CASE). As respostas articulam conceitos, exemplos e aplicações práticas, culminando na proposta de um sistema de agendamento de consultas.

A organização acompanha as cinco partes da lista fornecida. Os dois mapas mentais disponibilizados para a atividade foram utilizados como apoio temático, e a documentação técnica indicada nas referências fundamenta pontos específicos. Arquitetura de computadores, que envolve processador, memória e dispositivos, é distinguida de arquitetura de software, que trata da organização dos componentes e de suas interações.

Os cenários, dados cadastrais e resultados esperados são didáticos e fictícios. Os códigos são apresentados para estudo; os casos de teste descrevem verificações propostas, sem representar a homologação de um sistema real. A apresentação adota papel A4, margens superior e esquerda de 3 cm, inferior e direita de 2 cm, fonte Times New Roman de tamanho 12 no texto e espaçamento de 1,5. Códigos, legendas e referências utilizam espaçamento simples, com redução de fonte nos elementos técnicos quando necessária à legibilidade (ABNT, 2024).

2 ARQUITETURA DE SISTEMAS

Questão 01 - Arquitetura de software e sua importância

Arquitetura de software é o conjunto de decisões estruturais relevantes de um sistema: seus componentes, responsabilidades, interfaces, dependências e formas de comunicação. Ela também considera restrições e atributos de qualidade, como disponibilidade, segurança e capacidade de evolução.

Na manutenção, responsabilidades bem separadas permitem alterar uma regra sem modificar toda a aplicação. No desempenho, a arquitetura influencia o caminho das requisições, a localização dos dados e os pontos de contenção. Na evolução, interfaces estáveis facilitam acrescentar funções. Por exemplo, em uma biblioteca, separar a regra de empréstimo da tela permite oferecer um aplicativo móvel sem reescrever a mesma regra.

Questão 02 - Monólito, cliente-servidor e microsserviços

No monólito, os módulos principais são implantados como uma unidade. Sua vantagem é a simplicidade inicial de desenvolvimento e operação; sua limitação é que uma alteração normalmente exige publicar a aplicação inteira. Um sistema interno pequeno de controle de biblioteca pode usar essa abordagem.

Na arquitetura cliente-servidor, clientes solicitam serviços a um servidor, como um navegador acessando uma aplicação acadêmica. A vantagem é centralizar dados e regras; a limitação é a dependência do servidor e da conectividade. Esse estilo descreve uma relação de comunicação e pode coexistir com monólitos ou microsserviços.

Nos microsserviços, capacidades de negócio são organizadas em serviços com implantação independente e comunicação por contratos. A vantagem é permitir evolução e escala seletivas; a limitação é a complexidade de operação, observabilidade e consistência distribuída. Um delivery pode separar pedidos e pagamentos quando o porte e as necessidades justificarem.

Questão 03 - Responsabilidades das três camadas

A apresentação recebe entradas e exibe resultados: formulários, mensagens, navegação e estados de carregamento. Ela pode orientar o preenchimento, mas o servidor deve repetir as validações relevantes.

A camada de negócio aplica regras e coordena casos de uso. Em uma biblioteca, verifica a situação do leitor, a disponibilidade do exemplar e o limite de empréstimos. A persistência executa o acesso aos dados por repositórios e consultas, mapeando as informações

para o banco. O SGBD armazena e protege esses dados. Essa separação é lógica: três camadas não exigem três máquinas físicas.

Questão 04 - Escalabilidade vertical e horizontal

Escalabilidade vertical consiste em aumentar os recursos de uma instância, como memória e capacidade de processamento. É adequada para uma aplicação pequena cujo servidor apresenta falta de memória, desde que o investimento resolva o gargalo medido. Sua limitação é o teto do equipamento e, dependendo da operação, a necessidade de interrupção.

Escalabilidade horizontal consiste em acrescentar instâncias e distribuir a carga. É adequada para um portal de matrículas com muitos acessos simultâneos. Para funcionar bem, é preciso tratar sessões, estado compartilhado, balanceamento e capacidade do banco, pois replicar a aplicação não elimina automaticamente o gargalo de persistência.

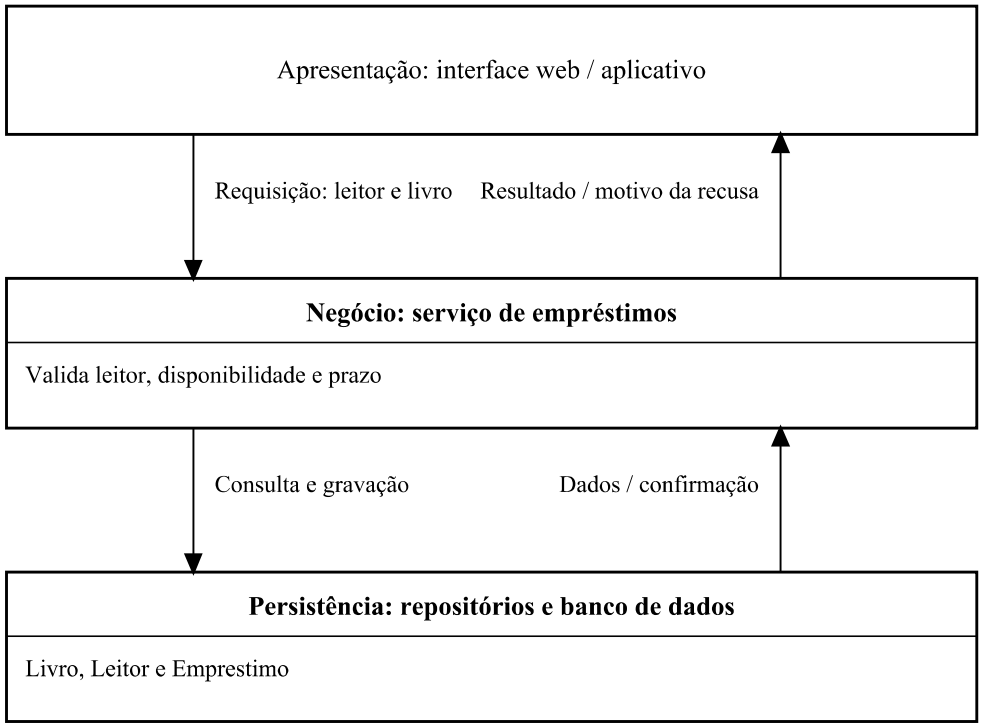
Questão 05 - Acoplamento e coesão

Acoplamento representa o grau de dependência entre módulos. Uma tela que conhece detalhes de tabelas e regras de pagamento tem acoplamento elevado. Coesão representa o quanto os elementos de um módulo contribuem para uma responsabilidade bem definida.

Baixo acoplamento reduz o impacto de mudanças e facilita testes isolados. Alta coesão facilita compreender e manter cada módulo. Por exemplo, um serviço de empréstimos deve reunir regras de retirada e devolução, enquanto um serviço de notificações trata do envio de avisos por uma interface definida.

Questão 06 - Arquitetura de uma biblioteca em três camadas

Figura 1 - Biblioteca em três camadas



Fonte: elaboração própria (2026).

A interface informa o identificador do leitor e do livro. O serviço consulta os dados, avalia as regras e registra o empréstimo em uma transação. A persistência devolve o resultado, e a interface apresenta a confirmação ou a justificativa da recusa. A interface não acessa o banco diretamente.

Questão 07 - Componentes de um aplicativo de delivery

Quadro 1 - Componentes do delivery

Componente	Responsabilidade
Aplicativo do cliente	Exibir estabelecimentos e cardápios; permitir montar e acompanhar o pedido.
API	Receber requisições, validar contratos e encaminhar casos de uso.
Autenticação e autorização	Identificar usuários e restringir ações conforme o perfil.
Catálogo	Gerenciar produtos, preços e disponibilidade informada pelo restaurante.
Pedidos	Calcular itens e taxas, registrar pedidos e controlar sua situação.
Pagamentos	Integrar o provedor, acompanhar confirmações e evitar cobranças duplicadas.
Entregas	Atribuir entregadores e atualizar o acompanhamento.
Notificações	Avisar mudanças relevantes, sem bloquear a operação principal.

Fonte: elaboração própria (2026).

Esses componentes podem ser módulos de um monólito. Sua identificação não exige transformá-los imediatamente em microsserviços. O módulo de pedidos deve registrar o preço aplicado no momento da compra, preservando o histórico mesmo quando o catálogo mudar.

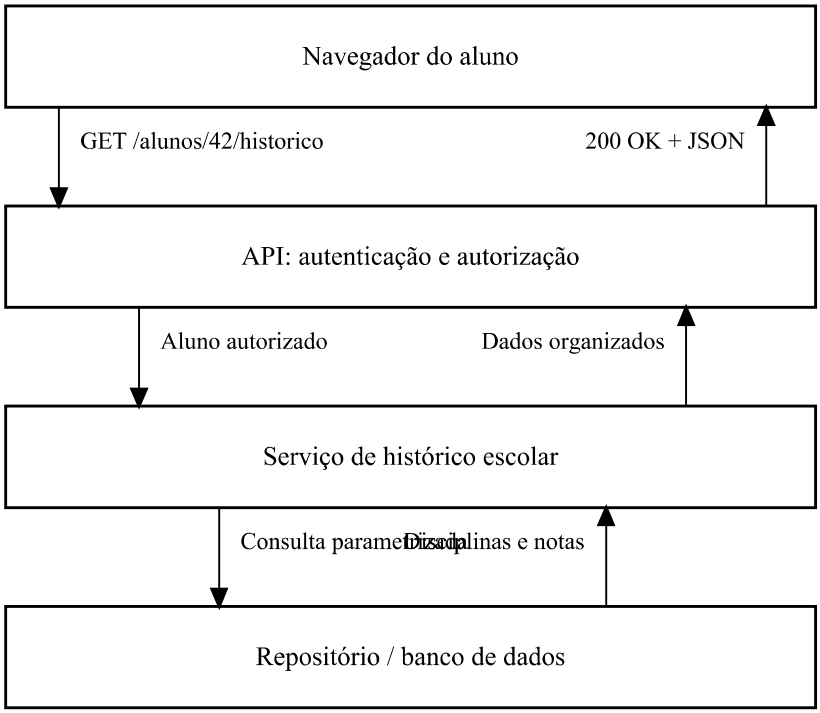
Questão 08 - Duas mudanças para picos de matrícula

1. Balancear a aplicação entre múltiplas instâncias. Colocar um balanceador na entrada e manter o estado de sessão fora da memória local de cada instância permite distribuir os acessos. A mudança amplia a capacidade da camada de aplicação e reduz a dependência de um único processo, desde que o banco e a infraestrutura suportem a carga.

2. Separar consultas de referência e a gravação da matrícula. Disponibilizar catálogo de disciplinas e horários em cache com expiração e invalidação reduz leituras repetidas. A confirmação da matrícula continua no banco autoritativo, com transação e controle de vagas. Assim, o cache alivia a consulta sem decidir a ocupação final. Antes e depois, devem ser medidos tempo de resposta, taxa de erros e contenção no banco; ganho de desempenho é uma expectativa a verificar.

Questão 09 - Fluxo HTTP de consulta ao histórico escolar

Figura 2 - Consulta autenticada ao histórico escolar



Fonte: elaboração própria (2026).

O navegador envia a requisição por HTTPS com a credencial de sessão. A API autentica o usuário e verifica se ele pode consultar o aluno indicado; conhecer o identificador não concede permissão. O serviço solicita os registros ao repositório, que usa consulta parametrizada. O resultado retorna como JSON para apresentação. Sessão inválida gera 401; falta de permissão, 403; falhas internas devem ser registradas sem expor detalhes sensíveis ao usuário.

Questão 10 - Comparação entre monólito e microsserviços

Quadro 2 - Comparação arquitetural

Critério	Monólito	Microsserviços
Implantação	Uma unidade principal; publicação coordenada.	Publicação independente, respeitando contratos.
Testes	Integração local mais simples; suíte pode crescer muito.	Exige testes de contrato e de integração distribuída.
Comunicação	Chamadas internas ao processo, em geral.	Rede e mensagens, com latência e falhas parciais.
Custos operacionais	Menor estrutura inicial de operação.	Mais serviços, monitoramento e automação.
Tolerância a falhas	Falha do processo pode afetar toda a aplicação.	Isolamento possível, se houver limites, timeouts e proteção contra cascatas.

Fonte: elaboração própria (2026).

3 PROCESSOS DE DESENVOLVIMENTO

Questão 11 - Etapas do desenvolvimento de software

Quadro 3 - Etapas e resultados

Etapa	Atividades e resultado
Levantamento de requisitos	Entrevistar interessados, observar processos e registrar necessidades e restrições.
Análise	Detalhar regras, esclarecer conflitos e modelar o problema e os casos de uso.
Projeto	Definir arquitetura, interfaces, dados e soluções técnicas.
Implementação	Codificar funcionalidades e integrar componentes de acordo com o projeto.
Testes	Verificar comportamento e qualidade, identificar defeitos e registrar evidências.
Implantação	Publicar de forma controlada, preparar dados, monitorar e prever reversão.
Manutenção	Corrigir falhas, adaptar regras e melhorar o sistema com base no uso.

Fonte: elaboração própria (2026).

As etapas podem ocorrer de forma iterativa. Testes, segurança e revisão de requisitos devem acompanhar o desenvolvimento, em vez de aparecerem apenas ao final.

Questão 12 - Cascata, incremental e espiral

O modelo cascata organiza o trabalho em fases predominantemente sequenciais. Atende melhor a contextos com requisitos estáveis e necessidade de marcos formais, mas mudanças tardias podem custar mais. Riscos podem ser analisados, embora o modelo não os organize como eixo central.

O incremental entrega partes utilizáveis em sucessivas versões. O retorno obtido em cada incremento ajuda a ajustar requisitos e reduzir incertezas. Exige priorização e uma arquitetura que permita integrar as novas partes.

O espiral organiza ciclos orientados à identificação e redução de riscos, combinando definição de objetivos, avaliação de alternativas, desenvolvimento e planejamento. Pode tratar mudanças e incertezas por protótipos e experimentos, mas demanda capacidade de gestão e pode ser excessivo para projetos muito simples.

Questão 13 - Requisitos funcionais e não funcionais de um hospital

Requisitos funcionais descrevem serviços e comportamentos: o que o sistema deve fazer. Requisitos não funcionais definem qualidades e restrições verificáveis: como e sob quais condições deve funcionar.

Quadro 4 - Exemplos para um sistema hospitalar

Tipo	Exemplo
Funcional 1	Cadastrar pacientes com identificação e contato.
Funcional 2	Agendar e cancelar consultas conforme disponibilidade.
Funcional 3	Registrar atendimento e permitir consulta ao prontuário por profissional autorizado.
Não funcional 1	Responder a 95% das consultas de agenda em até 2 s, com 200 usuários simultâneos no ambiente de referência.
Não funcional 2	Restringir prontuários por perfil e registrar acessos em trilha de auditoria.
Não funcional 3	Manter disponibilidade mensal de 99,5%, segundo janela e método de medição definidos.

Fonte: elaboração própria (2026).

Os valores são metas propostas para o exemplo, não medições de um sistema existente. Devem ser validados com os interessados e convertidos em critérios de verificação.

Questão 14 - Scrum, Kanban e DevOps

Scrum é um framework de trabalho iterativo com Sprints, objetivo, eventos de inspeção e adaptação e responsabilidades definidas. Seu foco é produzir incrementos úteis de valor e aprender com os resultados. Uma Sprint não é simplesmente uma lista fixa de tarefas sem objetivo (Schwaber; Sutherland, 2020).

Kanban concentra-se na gestão do fluxo: visualizar o trabalho, explicitar políticas, limitar trabalho em andamento e acompanhar indicadores como tempo de ciclo. Pode melhorar um processo já existente, sem exigir Sprints.

DevOps reúne cultura e práticas de colaboração entre desenvolvimento e operação, automação, integração e entrega contínuas e observabilidade. Seu foco é entregar e operar software com confiabilidade. As três abordagens podem se complementar; DevOps não se resume a uma ferramenta nem substitui a gestão do produto.

Questão 15 - Níveis de teste

O teste unitário verifica uma unidade pequena, como a função que calcula desconto, isolando dependências quando necessário. O teste de integração verifica a colaboração entre partes, como serviço de empréstimo e banco de dados.

O teste de sistema avalia a aplicação completa em um ambiente representativo, por exemplo, a jornada de matrícula. O teste de aceitação verifica se o comportamento atende aos critérios de negócio acordados, como impedir matrícula em turma sem vagas. O nível do teste depende do escopo e do objetivo, e não apenas da ferramenta empregada.

Questão 16 - Cinco histórias de usuário para um sistema acadêmico

Como aluno, quero consultar minhas notas por disciplina, para acompanhar meu desempenho.

Como aluno, quero solicitar matrícula em disciplinas disponíveis, para organizar meu semestre.

Como professor, quero registrar notas e frequências das minhas turmas, para atualizar os resultados acadêmicos.

Como coordenador, quero cadastrar turmas e vincular professores, para planejar a oferta do curso.

Como funcionário da secretaria, quero emitir declarações de matrícula para alunos autorizados, para atender às solicitações administrativas.

Cada história deve receber critérios de aceite. Por exemplo, o lançamento de nota deve aceitar apenas valores na escala definida pela instituição e restringir o professor às turmas sob sua responsabilidade.

Questão 17 - Backlog de cadastro de produtos

Quadro 5 - Backlog priorizado

ID	Tarefa	Prioridade	Esforço
P1	Definir campos, regras e critérios de aceite.	Alta	2 h
P2	Modelar tabela e restrições de produto.	Alta	3 h
P3	Criar operação de inclusão com validação.	Alta	5 h
P4	Construir formulário com mensagens de erro.	Alta	5 h
P5	Implementar listagem e pesquisa.	Alta	4 h
P6	Implementar edição e inativação.	Média	4 h
P7	Verificar duplicidade, preço inválido e permissões.	Alta	4 h
P8	Documentar uso e preparar publicação controlada.	Média	3 h

Fonte: elaboração própria (2026).

O esforço estimado total é de 30 horas de trabalho, sem representar prazo garantido. A ordem deve respeitar dependências: campos e modelo antes das operações; operações antes da integração da tela. A exclusão física foi substituída por inativação para preservar referências históricas.

Questão 18 - Sprint de duas semanas para uma tela de login

Objetivo: permitir que usuários cadastrados acessem a área restrita com autenticação segura e tratamento claro de falhas. Consideram-se dez dias úteis e um cadastro de usuários previamente disponível.

Entregas: formulário acessível, endpoint de autenticação, sessão protegida, encerramento de sessão, limitação de tentativas, estados de carregamento e erro e verificações automatizadas dos principais cenários. Nos dias 1 e 2, refinar contratos e critérios; de 3 a 6, implementar interface e serviço; de 7 a 9, integrar e testar; no dia 10, revisar o incremento e realizar a retrospectiva.

Critérios de aceite: credenciais válidas abrem a área autorizada; credenciais inválidas geram mensagem genérica; campos obrigatórios são validados também no servidor; senha não aparece em registros; a sessão expira e o logout a invalida; envios repetidos são controlados; a tela funciona por teclado e anuncia erros de forma compreensível.

Riscos: indisponibilidade do serviço de usuários, contrato indefinido, configuração incorreta de sessão e atraso na integração. A mitigação inclui definir o contrato no início, testar em ambiente separado e revisar o mecanismo de autenticação. A meta da Sprint deve ser renegociada com transparência caso uma dependência essencial impeça a entrega.

Questão 19 - Cinco casos de teste para recuperar senha

Quadro 6 - Recuperação de senha

Caso e pré-condição	Entrada / ação	Resultado esperado
T1: conta existente e e-mail de teste disponível.	Solicitar recuperação com o e-mail cadastrado.	Mensagem genérica; envio de link com token temporário; senha atual permanece válida até a troca.
T2: e-mail sem conta cadastrada.	Solicitar recuperação com endereço inexistente.	Mesma mensagem pública de T1; nenhuma conta criada e nenhum dado de usuário revelado.
T3: token válido e não utilizado.	Informar nova senha válida e confirmação igual.	Senha alterada com hash seguro; token invalidado; credencial antiga deixa de autenticar.
T4: token expirado ou já utilizado.	Tentar concluir a alteração.	Operação recusada; senha permanece inalterada; opção de solicitar novo link.
T5: token válido.	Enviar senha abaixo do mínimo ou confirmação divergente.	Servidor rejeita a entrada; nenhuma alteração; erro de validação compreensível.

Fonte: elaboração própria (2026).

Os casos são propostas de teste. Também convém verificar limites de solicitações e ausência de tokens nos logs. O sistema não deve enviar a senha anterior por e-mail nem armazenar senhas em texto puro.

Questão 20 - Quadro Kanban com dez tarefas

Quadro 7 - Kanban de um sistema de biblioteca

A fazer	Em andamento	Em teste	Concluído
1. Reservar livro	5. Registrar empréstimo	7. Pesquisar catálogo	9. Definir requisitos
2. Notificar atrasos	6. Registrar devolução	8. Cadastrar leitor	10. Modelar banco
3. Gerar relatório de uso			
4. Configurar backup			

Fonte: elaboração própria (2026).

O quadro representa um momento fictício do projeto. Propõem-se limites de duas tarefas em andamento e duas em teste. Uma tarefa só passa a concluída após atender aos critérios de aceite e à definição de pronto; novas tarefas devem ser puxadas conforme a capacidade, evitando acumular trabalho iniciado.

4 LINGUAGENS DE PROGRAMAÇÃO

Questão 21 - Compilação, interpretação e máquina virtual

Compilação transforma código-fonte em outra representação antes de sua execução, como código de máquina ou bytecode. Interpretação executa instruções por meio de um programa intermediário. A distinção descreve implementações e etapas, e não uma propriedade absoluta da linguagem.

Em uma implementação típica de C, o compilador gera código nativo. Java costuma ser compilado para bytecode executado pela JVM, que pode interpretá-lo e compilá-lo em tempo de execução. Em CPython, o código Python também passa por bytecode antes de ser executado pela máquina virtual. Motores modernos de JavaScript combinam estratégias de interpretação e compilação dinâmica. Portanto, plataformas podem usar as duas técnicas.

Questão 22 - Paradigmas procedural, orientado a objetos e funcional

Na programação procedural, a solução é organizada em procedimentos e sequências de comandos que manipulam estado. Uma rotina que percorre vendas e acumula o total é um exemplo.

Na orientação a objetos, estado e comportamento são organizados em objetos, com encapsulamento e colaboração. Uma classe Conta pode oferecer depositar e sacar, preservando regras de saldo.

Na programação funcional, destaca-se a composição de funções e a preferência por funções puras e dados imutáveis. Por exemplo, transformar uma lista de preços em outra lista de valores com desconto sem alterar a original. Muitas linguagens permitem combinar paradigmas conforme o problema.

Questão 23 - Tipagem estática e dinâmica

Na tipagem estática, a compatibilidade de tipos é verificada predominantemente antes da execução. Java e TypeScript são exemplos; a inferência pode dispensar declarações explícitas sem tornar a linguagem dinamicamente tipada.

Na tipagem dinâmica, verificações de tipo ocorrem durante a execução; Python e JavaScript são exemplos. Em Python, o nome de uma variável pode ser associado primeiro a um número e depois a uma string. Isso não significa ausência de tipos: os valores possuem tipos, e operações incompatíveis ainda podem falhar. Tipagem estática pode detectar certos erros antecipadamente; tipagem dinâmica oferece flexibilidade, exigindo atenção aos contratos e testes.

Questão 24 - HTML, CSS, JavaScript e back-end

HTML descreve a estrutura e a semântica da página, como títulos, campos e botões. CSS define a apresentação, incluindo cores, espaçamento, disposição e adaptação a telas. JavaScript implementa comportamentos interativos, como validar um campo e solicitar dados à API.

A linguagem de back-end, como Python ou Java, executa regras no servidor, controla acesso e integra o banco de dados. Em uma matrícula, HTML organiza o formulário, CSS o apresenta, JavaScript solicita disciplinas e o back-end verifica permissões e vagas. HTML é linguagem de marcação e CSS é linguagem de estilos; seus papéis não se confundem com o processamento de negócio.

Questão 25 - Variável, constante, função, classe e objeto

Variável é um nome associado a um valor que pode mudar no contexto permitido, como quantidade = 3. Constante representa uma vinculação que não deve ser reatribuída; detalhes dependem da linguagem. Em Python, TAXA = 0.10 é uma convenção, enquanto em JavaScript const impede reatribuir a variável, mas não torna automaticamente imutável o conteúdo de um objeto.

Função é uma unidade de comportamento que pode receber argumentos e retornar um resultado, como calcular_media(notas). Classe define uma estrutura de atributos e comportamentos para criar objetos. Objeto é uma instância concreta, como um Produto com código 1, nome Caderno e preço 25,00. Classes e objetos permitem reunir dados e operações relacionadas.

Questão 26 - Programa Python para média de três notas

```
from math import isfinite

notas = []
for numero in range(1, 4):
    while True:
        try:
            nota = float(input(f"Nota {numero}: ").replace(",", "."))
            if not isfinite(nota) or not 0 <= nota <= 10:
                raise ValueError
            notas.append(nota)
            break
        except ValueError:
            print("Informe uma nota numérica entre 0 e 10.")

media = sum(notas) / 3
print(f"Média: {media:.2f}")
print("Aprovado" if media >= 7 else "Reprovado")
```

Adota-se a escala de 0 a 10. A decisão usa a média calculada antes da formatação visual. Para 7, 8 e 6, a média esperada é 7,00 e o resultado é Aprovado. Para 6, 6 e 6, é 6,00 e Reprovado. Entradas inválidas são solicitadas novamente; valores não finitos, como NaN, são recusados.

Questão 27 - Função Python para total com desconto

```
from decimal import Decimal, InvalidOperation, ROUND_HALF_UP

def total_com_desconto(precos):
    total = Decimal("0")
    for preco in precos:
        try:
            valor = Decimal(str(preco))
        except InvalidOperation:
            raise ValueError("Preço inválido") from None
        if not valor.is_finite() or valor < 0:
            raise ValueError("Preço deve ser finito e não negativo")
        total += valor
    if total > Decimal("500"):
        total *= Decimal("0.90")
    return total.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

print(total_com_desconto(["300.00", "250.00"]))  # 495.00
print(total_com_desconto(["200.00", "300.00"]))  # 500.00
```

O desconto ocorre somente quando o total ultrapassa R$ 500,00; exatamente R$ 500,00 não recebe desconto. Uma lista vazia retorna 0,00. Decimal evita a representação binária aproximada usual de float em valores monetários; recomenda-se fornecer preços como strings decimais. Para o exemplo, o arredondamento monetário ocorre ao final.

Questão 28 - Validação de cadastro em JavaScript

```
function validarCadastro(dados) {
  const erros = {};
  const nome = String(dados?.nome ?? "").trim();
  const email = String(dados?.email ?? "").trim();
  const senha = String(dados?.senha ?? "");

  if (nome.length === 0) erros.nome = "Informe o nome.";
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
    erros.email = "Informe um e-mail com formato válido.";
  }
  if (senha.length < 8) {
    erros.senha = "A senha deve ter pelo menos 8 caracteres.";
  }
  return { valido: Object.keys(erros).length === 0, erros };
}

console.log(validarCadastro({
  nome: "Ana", email: "ana@example.com", senha: "Exemplo123"
})); // { valido: true, erros: {} }
```

A expressão regular realiza apenas uma verificação básica de formato, sem provar que a caixa de e-mail existe. O nome composto somente por espaços é rejeitado. A senha não é

aparada com trim, evitando alterar silenciosamente a entrada. A API deve repetir as validações e aplicar o mecanismo seguro de autenticação; a verificação no navegador serve à experiência de uso. As construções de funções, objetos e expressões seguem a linguagem documentada no guia JavaScript da MDN (Mozilla, [s. d.]).

Questão 29 - Pseudocódigo de busca de produto

```
FUNÇÃO buscarProduto(lista, codigoProcurado)
    PARA CADA produto EM lista FAÇA
        SE produto.codigo = codigoProcurado ENTÃO
            ESCREVA "Produto encontrado: ", produto.nome
            RETORNE produto
        FIM SE
    FIM PARA
    ESCREVA "Produto inexistente"
    RETORNE NULO
FIM FUNÇÃO
```

Trata-se de busca linear: o pior caso percorre n produtos, com complexidade O(n). A lista vazia resulta em Produto inexistente. Pressupõe-se que os códigos sejam únicos e comparados no mesmo tipo de dado; se houver duplicidade, esse algoritmo retorna a primeira ocorrência.

Questão 30 - Classe Produto em Python

```
from decimal import Decimal, ROUND_HALF_UP

class Produto:
    def __init__(self, codigo, nome, preco):
        self.codigo = codigo
        self.nome = nome.strip()
        self.preco = Decimal(str(preco))
        if not self.nome:
            raise ValueError("Nome obrigatório")
        if not self.preco.is_finite() or self.preco < 0:
            raise ValueError("Preço inválido")

    def preco_com_desconto(self, percentual):
        taxa = Decimal(str(percentual))
        if not taxa.is_finite() or not 0 <= taxa <= 100:
            raise ValueError("Desconto deve estar entre 0 e 100")
        valor = self.preco * (1 - taxa / Decimal("100"))
        return valor.quantize(Decimal("0.01"),
                              rounding=ROUND_HALF_UP)

produto = Produto(1, "Teclado", "200.00")
print(produto.preco_com_desconto("10"))  # 180.00
```

O método recebe o desconto em percentual: 10 representa 10%. O cálculo retorna um novo preço sem alterar o preço cadastrado. Desconto de 0% mantém o preço e desconto de 100% resulta em 0,00. A estrutura de classe, atributos de instância e métodos segue o modelo apresentado no tutorial da linguagem (Python Software Foundation, [s. d.]).

5 SISTEMAS DE GERENCIAMENTO DE BANCOS DE DADOS

Questão 31 - SGBD e conceitos fundamentais

SGBD é o software que administra a definição, o armazenamento, a consulta e a alteração de dados, oferecendo mecanismos de integridade, controle de acesso e recuperação. No modelo relacional, tabela organiza dados em linhas e colunas; registro corresponde a uma linha; campo é um atributo representado por uma coluna, ou seu valor em determinado registro.

Chave primária identifica unicamente cada registro e não aceita nulo. Chave estrangeira referencia uma chave candidata, normalmente a primária, de uma tabela relacionada. Por exemplo, Emprestimo.leitor_id referencia Leitor.id, vinculando cada empréstimo a um leitor existente.

Questão 32 - MySQL, Oracle Database e dBase

Quadro 8 - Contextos e recursos dos SGBDs

Sistema	História e aplicações	Modelo e recursos
MySQL	Surgiu em 1995 e difundiu-se em aplicações web e serviços. Possui edição comunitária e ofertas comerciais.	Relacional, cliente-servidor e SQL. Com InnoDB, oferece transações, chaves estrangeiras e controle de concorrência.
Oracle Database	Origem comercial no fim da década de 1970; uso frequente em aplicações corporativas e ambientes com exigências de administração e disponibilidade.	Relacional com recursos objeto-relacionais; SQL, PL/SQL, transações e administração avançada. Recursos variam por edição e licenciamento.
dBase	Popularizou-se nos microcomputadores na década de 1980; importante em aplicações departamentais e legadas do ecossistema xBase.	Dados tabulares em arquivos DBF, índices e linguagem xBase. Não equivale à arquitetura e ao conjunto de recursos de um servidor MySQL ou Oracle.

Fonte: elaboração própria (2026).

O formato DBF armazena descritores de campos e registros, como documentado pela dBASE ([s. d.]). A organização tabular do dBase não significa que todas as versões ofereçam as mesmas garantias, SQL, concorrência ou administração dos SGBDs servidores. A escolha deve considerar o contexto e a versão. A documentação da Oracle apresenta seu banco como sistema multiusuário com gerenciamento transacional (Oracle, [s. d.]a).

Questão 33 - Integridade referencial e restrições

Integridade referencial assegura que uma referência armazenada corresponda a um registro válido da tabela referenciada, conforme as regras definidas. Por exemplo, um empréstimo não pode apontar para um leitor inexistente.

NOT NULL impede ausência de valor em campos obrigatórios. UNIQUE impede repetição de uma chave ou combinação de valores, respeitando o tratamento de nulos do SGBD. FOREIGN KEY exige uma referência válida e define comportamentos de atualização e exclusão, como restringir a exclusão de leitor com empréstimos. Uma chave estrangeira que aceite nulo pode não exigir relacionamento; por isso, NOT NULL é necessário quando o vínculo é obrigatório.

Questão 34 - Propriedades ACID

Atomicidade: as operações de uma transação são confirmadas em conjunto ou desfeitas. **Consistência:** a transação preserva as regras e restrições definidas, desde que sua lógica seja correta. **Isolamento:** controla a interferência entre transações concorrentes; as garantias concretas dependem do nível de isolamento. **Durabilidade:** uma confirmação deve sobreviver a falhas dentro das garantias de persistência configuradas.

Em uma transferência de R$ 100,00, o débito de uma conta e o crédito de outra devem integrar a mesma transação. Se o crédito falhar, o débito é desfeito. Restrições e verificações preservam saldos permitidos; o isolamento impede interferências indevidas; a durabilidade protege o resultado confirmado. ACID não elimina erros nas regras implementadas nem substitui backup.

Questão 35 - Normalização e formas normais

Normalização organiza relações para reduzir redundâncias e anomalias de inserção, alteração e exclusão. As formas normais são avaliadas considerando chaves e dependências funcionais.

Na primeira forma normal (1FN), cada posição contém um valor do domínio e não há grupos repetidos. Em vez de armazenar vários telefones em um único campo, pode-se criar uma tabela relacionada. Na segunda forma normal (2FN), a relação está em 1FN e todo atributo não primo depende da totalidade de cada chave candidata, sem dependência parcial de parte de uma chave composta. Atributo primo é aquele que pertence a alguma chave candidata; não primo é aquele que não pertence a nenhuma delas.

Na terceira forma normal (3FN), para cada dependência funcional não trivial $X \rightarrow A$, X é superchave ou A é atributo primo. Em termos introdutórios, evita-se manter atributos não chave dependentes de outros atributos não chave. Por exemplo, Pedido(id, cliente_id, nome_cliente) repete informação do cliente; o nome deve ficar em Cliente. A normalização não deve ser confundida com dividir tabelas sem analisar as dependências.

Questão 36 - Modelo de biblioteca: Livro, Leitor e Emprestimo

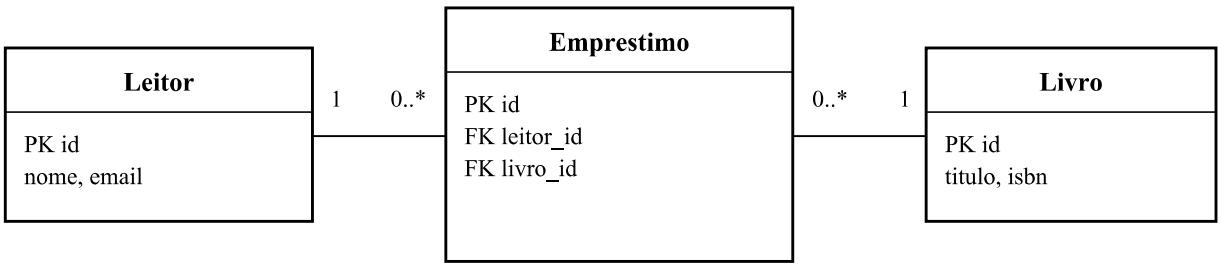
Quadro 9 - Entidades da biblioteca

Entidade	Atributos e restrições
Livro	id (PK), titulo, autor, isbn. Neste modelo, cada registro identifica um exemplar físico; isbn pode repetir.
Leitor	id (PK), nome NOT NULL, email UNIQUE NOT NULL, ativo NOT NULL.
Emprestimo	id (PK), livro_id (FK NOT NULL), leitor_id (FK NOT NULL), data_emprestimo, data_prevista, data_devolucao (opcional).

Fonte: elaboração própria (2026).

Um Leitor possui zero ou muitos empréstimos; cada Emprestimo pertence a exatamente um leitor. Um Livro possui zero ou muitos empréstimos ao longo do tempo; cada Emprestimo envolve exatamente um livro. O serviço deve impedir dois empréstimos ativos simultâneos do mesmo exemplar e exigir data prevista posterior ou igual à retirada. Para acervo maior, separar obra e exemplar permite representar várias cópias do mesmo título.

Figura 3 - Relacionamentos do modelo de biblioteca



Fonte: elaboração própria (2026).

Questão 37 - Criação da tabela Aluno em MySQL

```
CREATE TABLE Aluno (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    nome VARCHAR(150) NOT NULL,
    email VARCHAR(254) NOT NULL,
    data_nascimento DATE,
    CONSTRAINT pk_aluno PRIMARY KEY (id),
    CONSTRAINT uq_aluno_email UNIQUE (email)
) ENGINE = InnoDB
  DEFAULT CHARACTER SET = utf8mb4
  COLLATE = utf8mb4_0900_ai_ci;
```

O exemplo considera MySQL 8.0 ou 8.4, banco previamente selecionado e suporte à collation indicada. A chave primária é gerada automaticamente. E-mail obrigatório e UNIQUE impedem ausência e duplicidade segundo a comparação configurada. A data de

nascimento pode ser nula quando ainda não informada; a aplicação deve validar datas futuras e demais regras acordadas.

Questão 38 - Inserção de três alunos e consulta alfabética

```
INSERT INTO Aluno (nome, email, data_nascimento) VALUES
    ('Ana Lima', 'ana.lima@example.com', '2003-05-12'),
    ('Bruno Souza', 'bruno.souza@example.com', '2002-09-21'),
    ('Carla Dias', 'carla.dias@example.com', '2004-01-08');

SELECT nome
FROM Aluno
ORDER BY nome ASC, id ASC;
```

Considerando a tabela inicialmente vazia e somente esses registros, a saída esperada é Ana Lima, Bruno Souza e Carla Dias. ORDER BY define a ordenação; o id fornece desempate quando houver nomes iguais. Os endereços são fictícios, do domínio reservado example.com.

Questão 39 - Consulta com JOIN entre Cliente e Pedido

```
SELECT c.nome AS cliente, p.valor AS valor_pedido
FROM Cliente AS c
INNER JOIN Pedido AS p ON p.cliente_id = c.id
ORDER BY c.nome, p.id;
```

O INNER JOIN produz uma linha por pedido associado a um cliente. Um cliente com três pedidos aparece em três linhas. Clientes sem pedido ficam fora do resultado; caso fosse necessário incluí-los, seria apropriado LEFT JOIN, com valor de pedido nulo quando não houver correspondência.

Questão 40 - Concorrência na atualização de estoque

Pode ocorrer atualização perdida: dois operadores leem estoque 10 e cada um calcula uma saída, gravando um saldo baseado em leitura desatualizada. Também pode haver venda acima do disponível se a verificação e a baixa forem feitas separadamente sem proteção.

Uma estratégia é executar uma baixa condicional atômica em uma transação InnoDB. Para uma saída de três unidades do produto 42:

```
START TRANSACTION;

UPDATE Produto
SET estoque = estoque - 3
WHERE id = 42 AND estoque >= 3;

SELECT ROW_COUNT() AS linhas_atualizadas;
-- A aplicação lê o resultado imediatamente:
-- se for 1, registra o movimento e executa COMMIT;
-- se for 0, executa ROLLBACK e informa indisponibilidade.
-- Em caso de erro no registro do movimento, faz ROLLBACK.
```

O UPDATE protege a modificação concorrente do registro e avalia a condição no processamento da atualização. A quantidade deve ser positiva e o estoque obrigatório. Se for necessário ler vários campos antes da decisão, pode-se usar SELECT ... FOR UPDATE dentro da mesma transação; a documentação do MySQL descreve a leitura com bloqueio (Oracle, [s. d.]b). Transações devem ser curtas, e deadlocks ou tempo excedido precisam de tratamento e eventual repetição limitada. O trecho é um roteiro didático: não se deve deixar a transação aberta sem COMMIT ou ROLLBACK.

6 FERRAMENTAS CASE E PROJETO INTEGRADOR

Questão 41 - Definição e apoio das ferramentas CASE

CASE significa Computer-Aided Software Engineering, ou engenharia de software auxiliada por computador. São ferramentas que apoiam atividades do ciclo de vida por modelagem, documentação, geração, análise e manutenção de artefatos.

Na análise, ajudam a registrar requisitos e casos de uso. No projeto, apoiam diagramas de classes, arquitetura e dados. Na implementação, podem gerar estruturas iniciais, apoiar integração e analisar código. Na manutenção, permitem engenharia reversa e avaliação de impacto. A classificação tradicional distingue Upper CASE, voltado a análise e projeto; Lower CASE, ligado a implementação, testes e manutenção; e Integrated CASE, que integra diferentes fases. A abrangência real depende da ferramenta.

Questão 42 - Casos de uso, classes e sequência

O diagrama de casos de uso apresenta atores externos e objetivos atendidos pelo sistema, esclarecendo escopo e interação. Não deve ser usado como fluxograma detalhado da implementação.

O diagrama de classes representa a estrutura: classes, atributos, operações e relacionamentos, incluindo multiplicidades. O diagrama de sequência mostra a ordem temporal das mensagens entre participantes em um cenário. Os três oferecem visões complementares do comportamento e da estrutura; os diagramas deste trabalho empregam esses elementos da UML (Object Management Group, 2017).

Questão 43 - Modelagem, geração de código e engenharia reversa

Ferramentas de modelagem permitem construir e organizar representações, como um diagrama de classes. Geradores de código transformam modelos ou especificações em artefatos de implementação, como classes ou scripts SQL, que ainda exigem revisão e integração.

Ferramentas de engenharia reversa extraem representações a partir de algo existente, como gerar um modelo de tabelas de um banco. O resultado reflete a estrutura detectada, mas não recupera automaticamente todos os objetivos e regras de negócio. Uma mesma plataforma pode oferecer as três funções, com cobertura variável.

Questão 44 - Benefícios e limitações de CASE

Três benefícios: (1) padronizar representações e facilitar a comunicação da equipe; (2) reutilizar modelos e automatizar partes repetitivas da documentação; (3) apoiar a consistência entre requisitos, estrutura e implementação, facilitando revisões.

Duas limitações: (1) custo de licença, treinamento e manutenção dos modelos; (2) risco de documentação desatualizada ou geração de código inadequada quando a ferramenta é usada sem revisão. Ferramentas ajudam a registrar e verificar decisões, mas não substituem entendimento do problema nem asseguram qualidade por si mesmas.

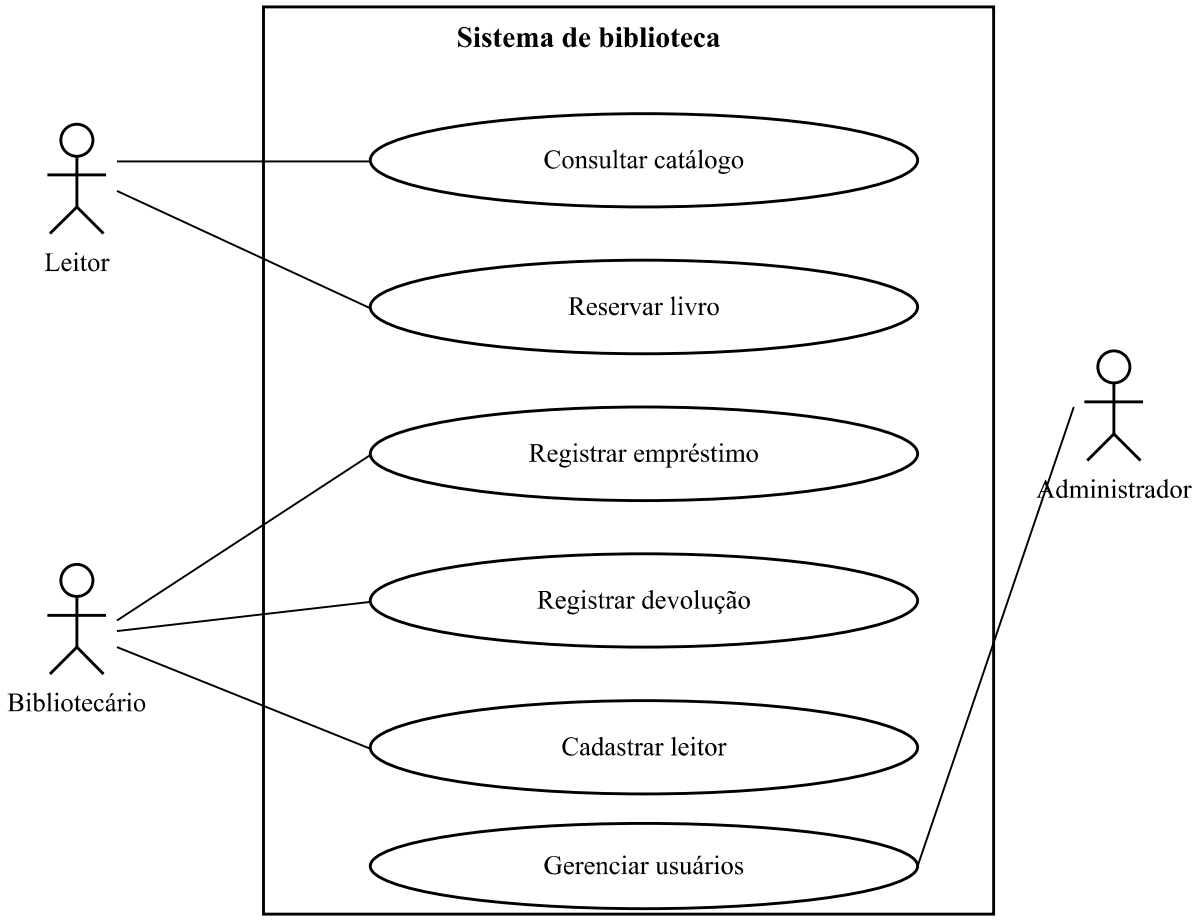
Questão 45 - Rastreabilidade e redução de retrabalho

Rastreabilidade relaciona cada requisito às decisões de projeto, aos componentes que o implementam e aos testes que o verificam. Uma documentação técnica concisa registra contratos, regras e justificativas importantes.

Por exemplo, RF-04 do projeto integrador: permitir apenas um agendamento ativo por horário relaciona-se ao serviço de agendamento, ao bloqueio transacional da linha de horário e ao teste T2 de duas reservas simultâneas. Se a regra mudar, esses vínculos ajudam a localizar os impactos. Isso reduz esquecimentos e redescobertas, desde que os registros sejam mantidos junto às mudanças.

Questão 46 - Diagrama de casos de uso de biblioteca

Figura 4 - Casos de uso do sistema de biblioteca

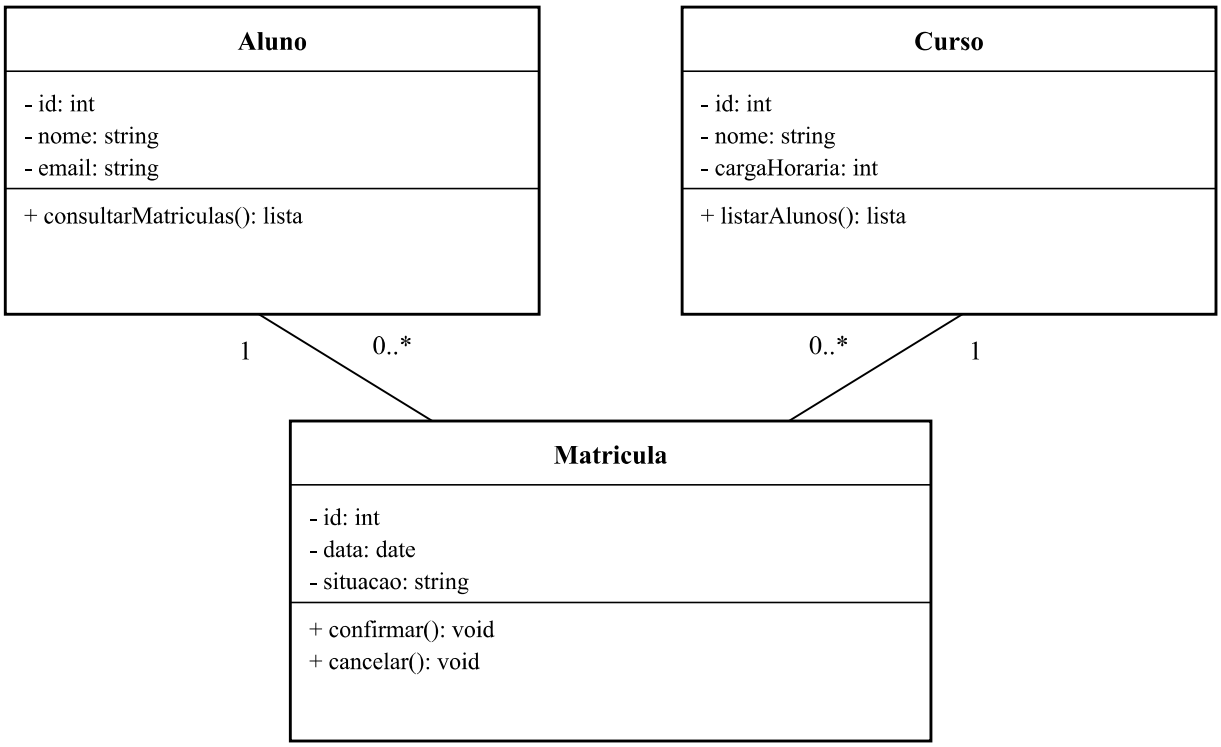


Fonte: elaboração própria (2026).

São três atores e seis casos de uso. Leitor consulta o catálogo e reserva livro; Bibliotecário registra empréstimo e devolução e cadastra leitor; Administrador gerencia usuários e permissões. Os atores permanecem fora do limite do sistema. Autenticação é uma precondição dos casos restritos, sem acrescentar um sétimo caso de uso ao escopo solicitado. O arquivo editável de diagramas que acompanha o PDF pode ser aberto no diagrams.net.

Questão 47 - Diagrama de classes Aluno, Curso e Matricula

Figura 5 - Classes do sistema acadêmico

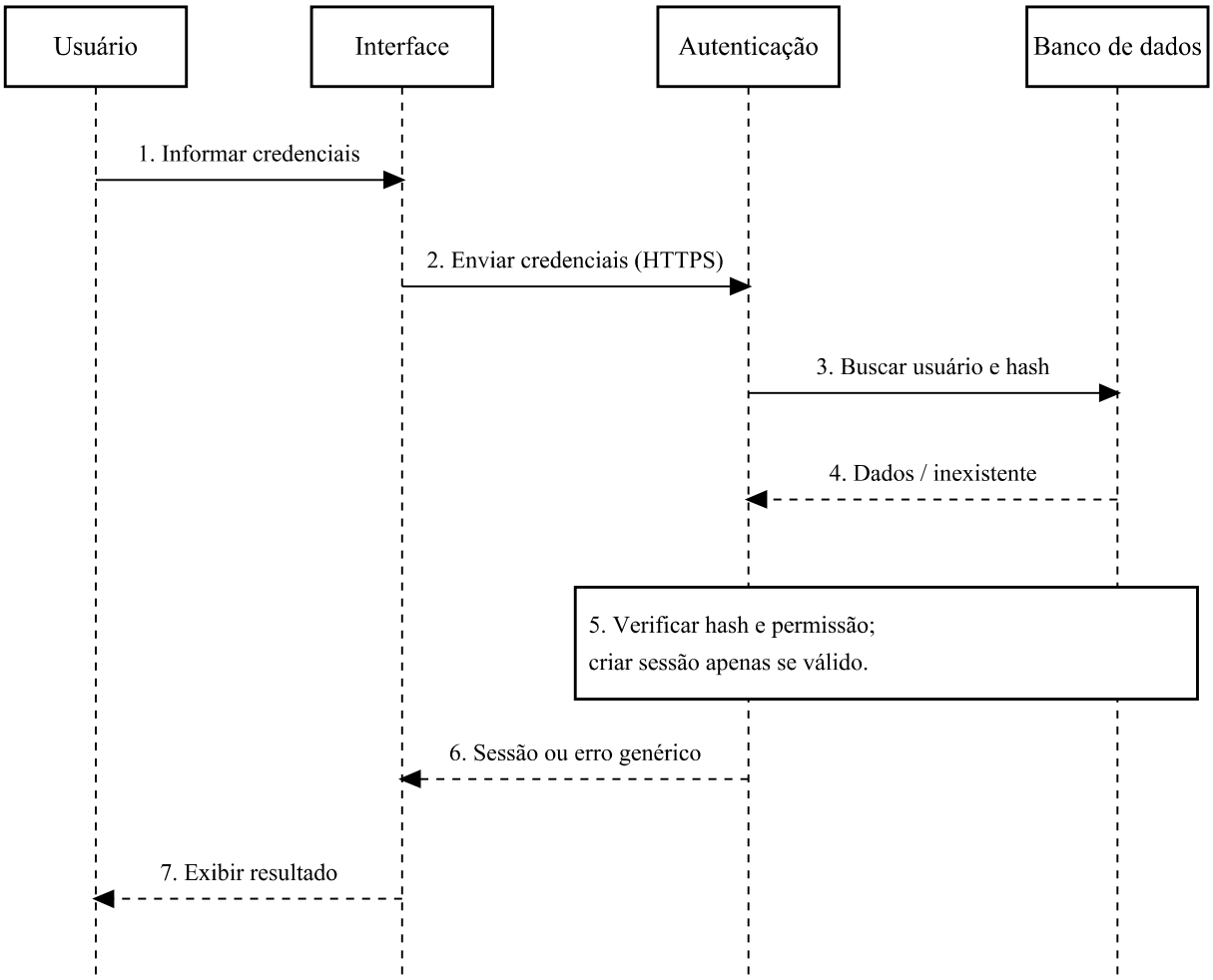


Fonte: elaboração própria (2026).

Cada Matricula pertence a um Aluno e a um Curso, ambos obrigatórios. Um Aluno pode ter zero ou muitas matrículas, e um Curso pode ter zero ou muitas matrículas. Matricula resolve a associação entre alunos e cursos e guarda dados próprios do vínculo. Os sinais - e + representam, respectivamente, visibilidade privada e pública. Neste escopo, deve-se impedir duplicidade de matrícula ativa do mesmo aluno no mesmo curso.

Questão 48 - Diagrama de sequência do login

Figura 6 - Sequência de autenticação de usuário

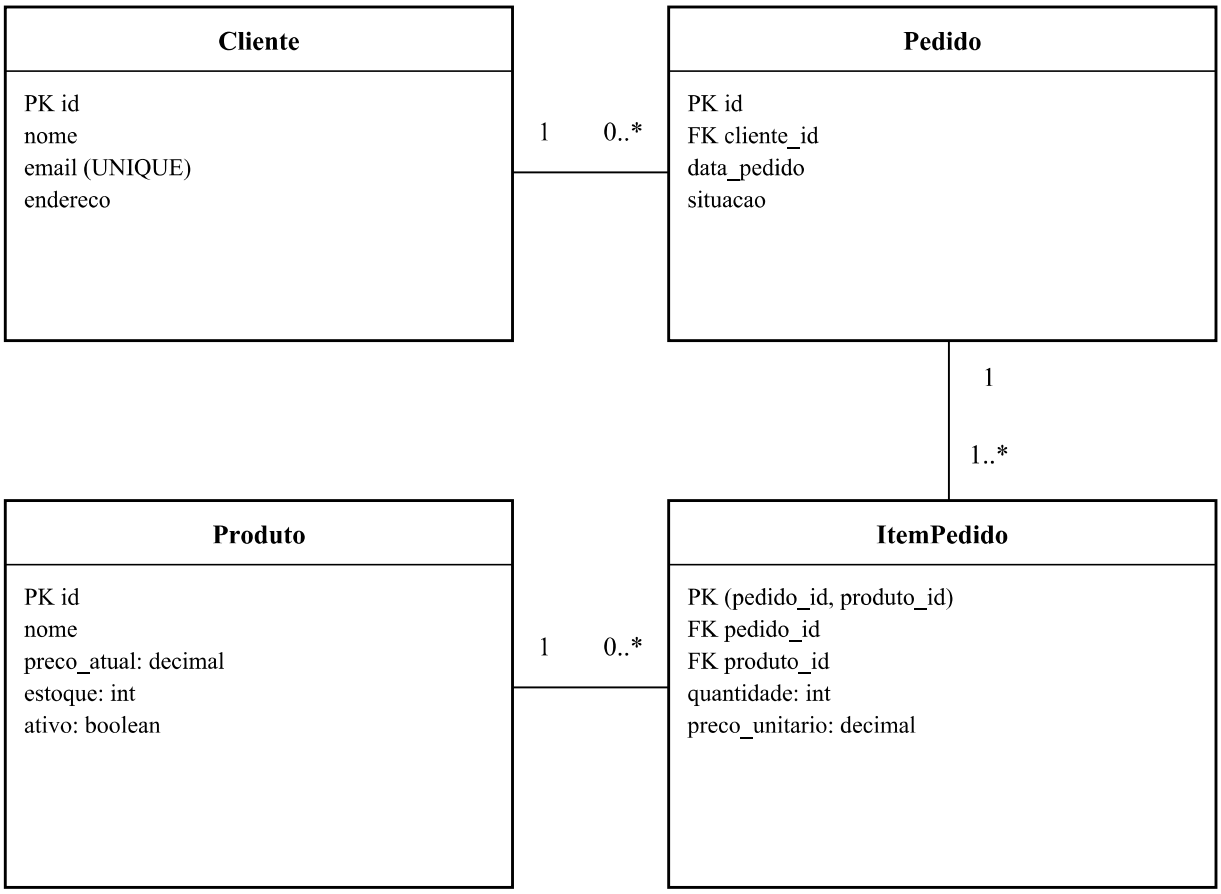


Fonte: elaboração própria (2026).

O serviço verifica o hash da senha com uma biblioteca apropriada; não compara nem recupera senha em texto puro. No sucesso, cria a sessão e a interface abre a área autorizada. Em falha, responde com mensagem genérica para usuário inexistente ou senha incorreta. Limitação de tentativas e controle de expiração completam o fluxo. A senha não deve ser registrada nos logs.

Questão 49 - Diagrama entidade-relacionamento de loja virtual

Figura 7 - Modelo lógico relacional da loja virtual



Fonte: elaboração própria (2026).

Um Cliente tem zero ou muitos pedidos, e cada Pedido pertence a um cliente. Um Pedido confirmado contém um ou muitos itens; cada ItemPedido pertence a um pedido e referencia um produto. Um Produto pode aparecer em zero ou muitos itens. A chave composta impede repetir o mesmo produto no pedido; quantidades repetidas devem ser somadas na linha existente.

O preço unitário do item registra o valor da venda, preservando o histórico. Quantidade deve ser positiva e valores monetários, não negativos. O total do pedido pode ser calculado pela soma de quantidade × preço unitário. A exigência de ao menos um item em pedido confirmado depende de validação transacional; uma chave estrangeira isolada não garante essa cardinalidade mínima. O modelo é fornecido também em formato editável compatível com diagrams.net.

Questão 50 - Projeto integrador: sistema de agendamento de consultas

Propõe-se o sistema Agenda Clínica para uma clínica com consultas de 30 minutos e horários previamente cadastrados em uma grade sem sobreposição por profissional. Pacientes solicitam e cancelam agendamentos; recepcionistas administram a agenda; profissionais consultam seus atendimentos. O escopo não inclui prontuário clínico, prescrição, pagamentos ou teleconsulta.

a) Cinco requisitos funcionais e três não funcionais

Quadro 10 - Requisitos funcionais do Agenda Clínica

ID	Requisito e critério de aceite
RF-01	Cadastrar pacientes: exigir nome e e-mail, atribuir identificador e rejeitar e-mail já cadastrado.
RF-02	Cadastrar profissionais e horários de 30 minutos em grade sem sobreposição: impedir horários duplicados ou conflitantes para o mesmo profissional.
RF-03	Consultar disponibilidade por profissional e data: listar apenas horários futuros livres.
RF-04	Agendar consulta: validar paciente e horário e permitir apenas um agendamento ativo por horário.
RF-05	Cancelar consulta autorizada e consultar agenda: registrar cancelamento sem apagar histórico e liberar o horário na mesma transação.

Fonte: elaboração própria (2026).

Quadro 11 - Requisitos não funcionais do Agenda Clínica

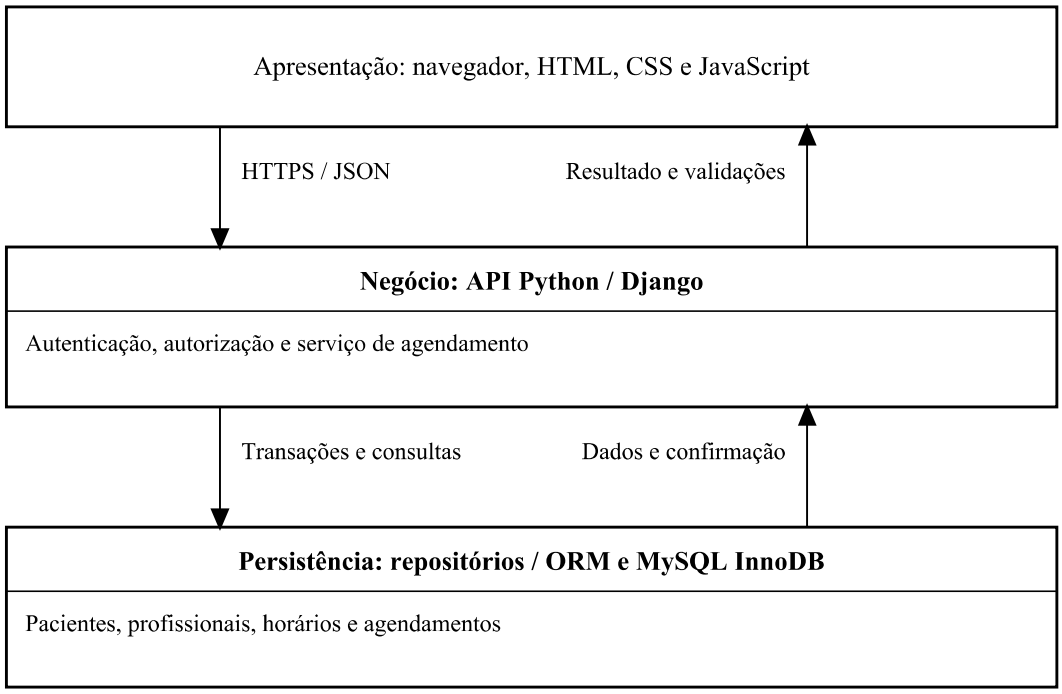
ID	Meta verificável proposta
RNF-01	Segurança: autenticação para ações restritas, autorização por perfil e titularidade, HTTPS e registro auditável de alterações; nenhum usuário acessa agenda de outro paciente sem permissão.
RNF-02	Desempenho: 95% das consultas de disponibilidade em até 2 s com 100 usuários simultâneos, em ambiente e massa de dados definidos para o teste.
RNF-03	Recuperação: backup diário, perda máxima de 24 h de dados (RPO) e restauração em até 4 h (RTO), comprovadas em ensaio de restauração.

Fonte: elaboração própria (2026).

Os requisitos de qualidade são metas do projeto. Seu cumprimento depende de implementação, configuração e testes, e não é afirmado neste documento.

b) Arquitetura em três camadas

Figura 8 - Arquitetura proposta para o Agenda Clínica



Fonte: elaboração própria (2026).

A aplicação pode ser implantada como monólito modular, mantendo a separação lógica das três camadas. O servidor é a autoridade para disponibilidade e permissões. Não há necessidade inicial de microsserviços: o porte e o escopo propostos favorecem uma operação simples.

c) Linguagem e justificativa

Escolhe-se Python para o back-end por sua legibilidade e pelo ecossistema de desenvolvimento web. Django é uma opção de framework com recursos para organização, acesso a dados, autenticação e testes. A interface utiliza HTML, CSS e JavaScript, e os dados ficam em MySQL com InnoDB. A escolha favorece a implementação de um protótipo acadêmico; desempenho e facilidade de manutenção ainda precisam ser verificados no uso concreto.

d) Modelo de dados e regras de integridade

Quadro 12 - Entidades do Agenda Clínica

Entidade	Atributos principais
Paciente	id (PK), nome (NOT NULL), email (NOT NULL, UNIQUE), telefone.
Profissional	id (PK), nome, especialidade, registro_profissional (UNIQUE).
Horario	id (PK), profissional_id (FK), inicio, fim, agendamento_ativo_id (FK opcional UNIQUE). UNIQUE(profissional_id, inicio).
Agendamento	id (PK), paciente_id (FK), horario_id (FK), criado_em, status (AGENDADO, CANCELADO ou REALIZADO).
Usuario	id (PK), login (UNIQUE), senha_hash, perfil, paciente_id (FK opcional UNIQUE), profissional_id (FK opcional UNIQUE).

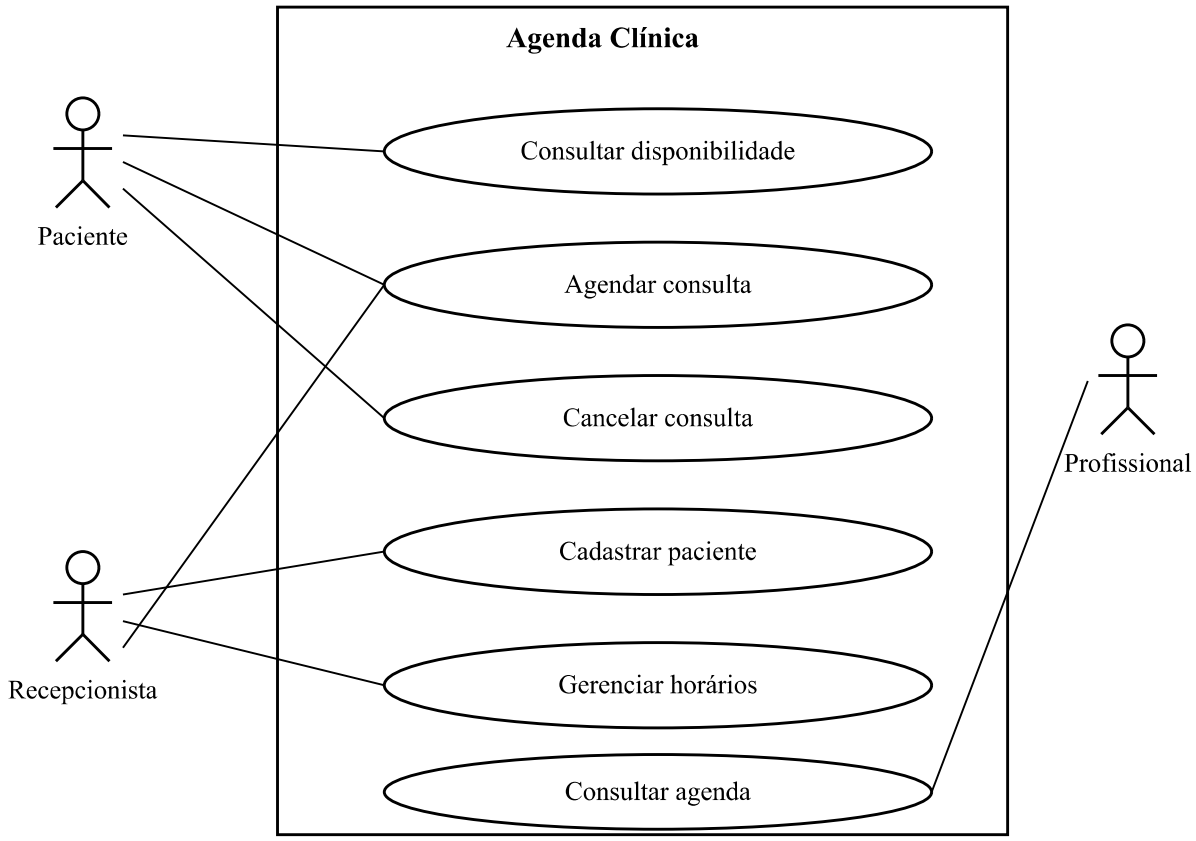
Fonte: elaboração própria (2026).

Cada Horario pertence a um Profissional. Cada Agendamento referencia exatamente um Paciente e um Horario. Um horário mantém o histórico de zero ou muitos agendamentos, mas aponta para no máximo um agendamento ativo. O vínculo entre Horario.agendamento_ativo_id e Agendamento deve ser validado para que o agendamento pertença ao próprio horário. Usuario representa a identidade de acesso; vínculos opcionais distinguem paciente, profissional e recepcionista.

Para reservar, o serviço abre uma transação, bloqueia a linha de Horario com SELECT ... FOR UPDATE, verifica se o horário é futuro e está sem agendamento ativo, cria Agendamento e atualiza o ponteiro ativo antes de confirmar. O cancelamento usa o mesmo bloqueio, registra o estado e limpa o ponteiro na mesma transação. Assim, uma nova reserva não apaga o histórico. Todos os caminhos de gravação devem obedecer a esse protocolo. Como a duração é fixa e a agenda é criada em slots sem sobreposição, a reserva disputa um único horário; permitir durações variáveis exigiria outra regra de conflito.

e) Dois diagramas CASE

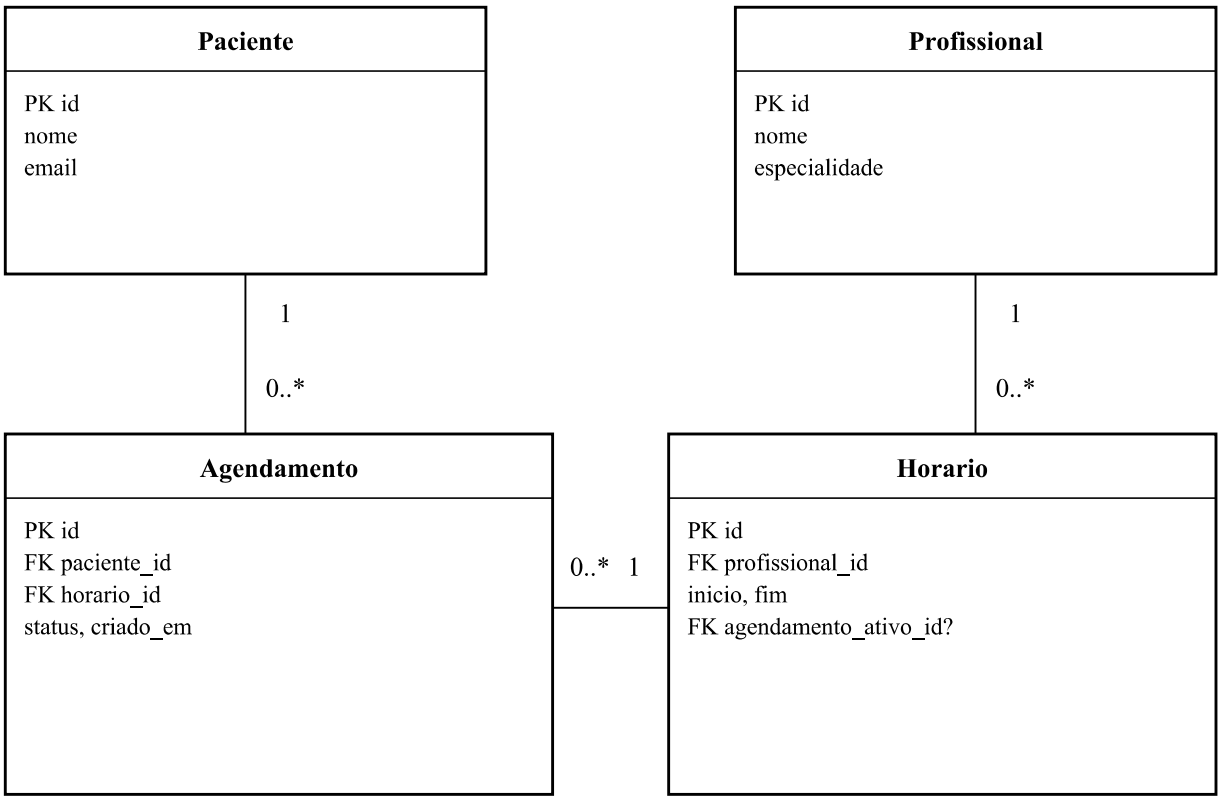
Figura 9 - Casos de uso do Agenda Clínica



Fonte: elaboração própria (2026).

O diagrama apresenta os objetivos de cada perfil. A recepção pode operar em nome do paciente conforme autorização; o profissional consulta sua própria agenda. Os demais detalhes de permissão devem ser especificados antes da implementação.

Figura 10 - Modelo de dados do Agenda Clínica



Histórico por horário; vínculo ativo opcional descrito no texto.

Fonte: elaboração própria (2026).

A Figura 10 mostra as quatro entidades centrais e as cardinalidades do histórico. Usuario, detalhado no Quadro 12, complementa o controle de acesso e foi omitido da figura para preservar a leitura. O ponteiro ativo é uma referência adicional: um Horario pode apontar para zero ou um Agendamento ativo; um Agendamento pode ser o ativo de zero ou um Horario, obrigatoriamente o seu próprio horário.

f) Cinco testes e plano de quatro semanas

Quadro 13 - Testes propostos para o Agenda Clínica

Teste / preparação	Ação	Resultado esperado
T1 - paciente cadastrado e horário futuro livre.	Agendar com usuário autorizado.	Um agendamento AGENDADO; horário passa a apontar para ele; confirmação exibida.
T2 - duas sessões autorizadas e mesmo horário livre.	Enviar duas reservas simultaneamente.	Apenas uma é confirmada; a outra recebe indisponibilidade; não há duplicidade.
T3 - paciente autenticado, horário passado ou ID inexistente.	Tentar agendar.	Rejeição pela regra ou registro inexistente; nenhuma alteração parcial.
T4 - agendamento ativo do próprio paciente.	Cancelar e repetir o cancelamento.	Cancelamento registrado uma vez; horário livre; repetição não corrompe o histórico.
T5 - paciente A autenticado e agendamento de B.	Tentar ler ou cancelar usando ID de B.	Acesso negado sem exposição de dados; agendamento de B inalterado.

Fonte: elaboração própria (2026).

Quadro 14 - Plano de desenvolvimento de quatro semanas

Semana	Atividades e entrega verificável
1 - definição	Validar escopo e critérios; desenhar interface e modelos; planejar testes. Entrega: requisitos aprovados, protótipo e modelo de dados revisado.
2 - base funcional	Implementar autenticação, perfis, cadastros e horários com restrições. Entrega: ambiente de teste com cadastro e consulta de disponibilidade integrados.
3 - agendamento	Implementar reserva e cancelamento transacionais; executar testes de concorrência e autorização. Entrega: fluxo completo e registro de resultados dos testes.
4 - validação e entrega	Realizar aceitação, testes de carga e restauração, corrigir defeitos e documentar operação. Entrega: versão candidata, evidências e plano de implantação e reversão.

Fonte: elaboração própria (2026).

As quatro semanas constituem uma estimativa acadêmica condicionada à capacidade da equipe e às integrações disponíveis. A publicação deve ocorrer após os testes previstos e a validação dos responsáveis. Segurança, concorrência e recuperação são critérios de entrega, não atividades opcionais.

7 CONSIDERAÇÕES FINAIS

Os exercícios mostram que sistemas de informação dependem de decisões integradas: arquitetura organiza responsabilidades, processos orientam o trabalho, linguagens expressam comportamentos, SGBDs preservam dados e ferramentas CASE tornam os modelos mais claros. A qualidade surge da coerência entre esses elementos e de sua verificação.

O projeto Agenda Clínica reúne essa visão em um escopo reduzido e verificável. A separação em camadas, o controle transacional dos horários, as permissões explícitas e os testes de cenários concorrentes são escolhas proporcionais ao problema. As propostas apresentadas devem ser refinadas conforme os requisitos reais e validadas antes de qualquer uso em produção.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 14724:2024**: informação e documentação: trabalhos acadêmicos: apresentação. Versão corrigida em 1 abr. 2025. Rio de Janeiro: ABNT, 2024. Disponível em: https://engenhariaedesenvolvimentosustentavel.ufes.br/sites/engenhariaedesenvolvimentosustentavel.ufes.br/files/field/anexo/abnt_nbr_14724.pdf. Acesso em: 22 out. 2026.

DBASE. **dBASE .DBF file structure**. [S. l.]: dBASE, [s. d.]. Disponível em: https://www.dbase.com/Knowledgebase/INT/db7_file_fmt.htm. Acesso em: 22 out. 2026.

MAPA mental: desenvolvimento de software e bancos de dados. [S. l.: s. n.], [s. d.]. 1 imagem digital. Material de apoio fornecido pelo estudante; autoria não identificada.

MAPA mental de sistemas computacionais e informação. [S. l.: s. n.], [s. d.]. 1 imagem digital. Material de apoio fornecido pelo estudante; autoria não identificada.

MOZILLA. **JavaScript guide**. MDN Web Docs. [S. l.]: Mozilla, [s. d.]. Disponível em: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide. Acesso em: 22 out. 2026.

OBJECT MANAGEMENT GROUP. **Unified Modeling Language**: version 2.5.1. [S. l.]: OMG, 2017. Disponível em: https://www.omg.org/spec/UML/2.5.1. Acesso em: 22 out. 2026.

ORACLE. **Introduction to Oracle Database**. Oracle Database 19c: Database Concepts. [S. l.]: Oracle, [s. d.]a. Disponível em: https://docs.oracle.com/en/database/oracle/oracle-database/19/cncpt/introduction-to-oracle-database.html. Acesso em: 22 out. 2026.

ORACLE. **MySQL 8.4 reference manual**: locking reads. [S. l.]: Oracle, [s. d.]b. Disponível em: https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html. Acesso em: 22 out. 2026.

PYTHON SOFTWARE FOUNDATION. **The Python tutorial**. [S. l.]: Python Software Foundation, [s. d.]. Disponível em: https://docs.python.org/3/tutorial/. Acesso em: 22 out. 2026.

SCHWABER, Ken; SUTHERLAND, Jeff. **The Scrum guide**: the definitive guide to Scrum: the rules of the game. [S. l.]: Scrum Guides, 2020. Disponível em: https://scrumguides.org/scrum-guide.html. Acesso em: 22 out. 2026.

50 EXERCÍCIOS conceituais e práticos: arquitetura e processos de desenvolvimento, linguagens de programação, SGBDs, ferramentas CASE. [S. l.: s. n.], [s. d.]. 4 p. Material didático fornecido pelo estudante; autoria não identificada.